TEMPLE COLLEGE
WORKFORCE DEVELOPMENT

CHIEFOS

STUDENT REVIEW PACKET

Week 15

Inline edit, agent mode — Module 05 checkpoint

Phase 3 — AI-Assisted Development · Weeks 13–18 · Modules 05–06 + Capstone

No D2L login required. This is the same lesson text posted in Brightspace for this week. Work the lessons in order on your own machine. Keep each checkpoint screenshot or pasted output. Bring it to the live session, or email it to your instructor, instead of submitting in D2L. Module quizzes stay in D2L and are not included here.

In this packet

1. 05.07 Inline Edit
2. 05.08 Agent Mode
3. 05.09 Final Checkpoint

Learning objective: Use `Ctrl+I` inline edits to rename a function, add validation, extract a helper, and draft a test, reviewing every proposed diff before accepting, rejecting, or partially accepting it.

Lesson 07: Inline Edit

By the end, you will have used `Ctrl+I` to highlight code and ask the model to change it — accepting, rejecting, and partially accepting the diff.

~15 minutes of typing + reading diffs

> Inline edit (`Ctrl+I`) is the second-most-used Continue feature. You select code, describe a change, the model proposes a diff, and you accept or reject it line by line.

---

1. Open `routers/items.py` in VS Code.

2. Confirm clean state.

```
cd ~/docker-course/fastapi-app && git status
```

Drill 1: rename a function

3. Find the `read_item` function. **Highlight the entire function** (from `@router.get(...)` decorator through the end of the body).

4. Press **`Ctrl+I`** (Cmd+I on Mac). A small input box appears at the top of the editor.

5. Type:

```
Rename this function from read_item to get_item_by_id. Update the function name
only — keep the route path and behavior identical.
```

6. Press Enter.

7. A diff overlay appears in your editor. Red = removed, green = added. Read it carefully.

8. If it looks right, click **Accept** (or press the accept shortcut shown — usually `Ctrl+Enter`).

9. If it looks wrong, click **Reject** (`Esc` or `Ctrl+Z` after accepting).

10. You can also **accept partial** — accept some hunks of the diff and reject others. Look for the per-hunk buttons.

11. Confirm the function was renamed.

```
grep -n "def get_item_by_id\|def read_item" routers/items.py
```

You should see the new name and not the old one.

Drill 2: add error handling

12. Find the `update_item` function you wrote in lesson 06. Highlight the **body** of the function (the four-or-so lines after the `def` line).

13. `Ctrl+I`. Type:

```
Add a check at the start of the body: if item.price is provided and is <= 0, raise
HTTPException 400 with the message "price must be positive". Keep the rest of the
logic the same.
```

14. Read the proposed diff. The model should add a 2–3 line check before the existing logic.

15. Accept if it's right; reject and re-prompt if not.

16. Confirm.

```
grep -B 1 "price must be positive" routers/items.py
```

Drill 3: extract a helper

17. Highlight the `db.query(models.Item).filter(models.Item.id == item_id).first()` line that appears in `read_item` / `get_item_by_id`, `update_item`, and `delete_item` — pick **one occurrence**.

18. `Ctrl+I`. Type:

```
This pattern repeats. Extract it into a helper function `get_item_or_404(db,
item_id)` defined at the top of the file (after imports, before the router) that
does the query and the 404 check. Then use the helper here. Don't change the other
call sites yet.
```

19. This is a bigger change. The model may propose multi-line changes. Read carefully.

20. Accept. Now manually update the **other** two call sites (`update_item`, `delete_item`) to use the helper. You can do each with `Ctrl+I` again — highlight just the query+404 lines and ask the model to "use the get_item_or_404 helper instead."

21. Run a quick syntax check:

```
python3 -c "import ast; ast.parse(open('routers/items.py').read()); print('OK')"
```

You should see `OK`.

22. Restart the stack and confirm endpoints still work.

```
docker compose restart api
sleep 3
curl -s -o /dev/null -w "%{http_code}\n" http://localhost:8000/items
```

Should print `200`.

Drill 4: write a test

23. Open a fresh terminal.

```
cd ~/docker-course/fastapi-app
source .venv/bin/activate
pip install pytest httpx
```

24. Create a test file.

```
mkdir -p tests
touch tests/__init__.py
touch tests/test_items.py
```

25. Open `tests/test_items.py`. Type only:

```
from fastapi.testclient import TestClient

from main import app


client = TestClient(app)
```

Save.

26. Highlight the whole file. `Ctrl+I`. Type:

```
Add a pytest test function called test_create_and_read_item that:
1. POSTs a new item to /items with name "Test" and price 1.0
2. asserts the response status is 201
3. extracts the id from the response
4. GETs /items/{id}
5. asserts the response status is 200 and the name matches.
```

27. Read the diff. Accept if reasonable.

28. Run the test (note: this runs the app in-process against an in-memory or whatever DB the test client uses — it'll likely fail if your code requires the real Postgres. That's fine for the drill, you saw the model write a real test).

```
pytest tests/test_items.py -v 2>&1 | tail -n 20
```

29. Don't worry about test pass/fail right now — getting the test infrastructure wired up properly is its own module. The point is you wrote a real test by describing it.

Drill 5: clean up + commit

30. Deactivate venv (we're done with the test).

```
deactivate
```

31. Commit the helper extraction.

```
git add routers/items.py
git commit -m "Extract get_item_or_404 helper"
```

32. The test file is in-progress; leave it for now or commit it as a WIP.

```
git add tests/
git commit -m "WIP: add first integration test"
```

33. Log it.

```
echo "Inline edit drilled: rename, add validation, extract helper, write test" >>
~/docker-course/notes/notes.txt
```

Watch out

- **Always read the diff.** Inline edit produces real changes to your file. Accept blindly and you'll spend an hour debugging "why does my route return 500." Habit: read every red and green line.
- **Smaller selections = better edits.** Highlighting a whole 200-line file and saying "fix all the things" leads to chaos. Pick the smallest scope that contains the change you want.
- **Imports get missed sometimes.** If the model adds a function call to a new module, it might not add the `import`. Run a syntax check or eyeball the imports.
- **Use `Ctrl+Z` if you accepted by accident.** Continue's edits are normal VS Code undo entries — one undo reverts the whole change.

Checkpoint

```
grep "get_item_or_404" routers/items.py | wc -l
```

Should print at least `2` (definition + at least one call).

```
grep -c "@router\." routers/items.py
```

Should match the number of routes you have (at least 5).

```
git log --oneline -2
```

Should show your "Extract get_item_or_404 helper" commit.

Next up: 08-agent-mode.md — Let the model run commands and edit multiple files

Lab — do it for real. Complete every step above on your own machine — type the commands, don't copy-paste. **Submit:** a screenshot or pasted output of the *Checkpoint* section (or this lesson's final verification steps) Keep it. If you cannot submit in D2L, bring it to class or email it to your instructor. Completing the lab unlocks the next lesson.

Learning objective: Use Continue's agent mode to make approval-gated, multi-file changes on a feature branch, reviewing each tool call and rejecting steps that are wrong or destructive.

Lesson 08: Agent Mode

By the end, you will have used Continue's agent mode to make a multi-file change with shell command tool calls — accepting and rejecting individual steps along the way.

~25 minutes

> **Agent mode** lets the model do more than chat or edit one selection. It can list files, read files, write files, and run shell commands — chaining tool calls to complete a higher-level task. **Every tool call is gated on your approval.** You watch what it wants to do, you approve or reject, you keep control.

Concept

Three modes Continue supports (toggled at the bottom of the chat panel):

Mode	What the model can do
Chat	Read files you `@`-reference. No edits. No commands. (Lesson 05.)
Edit	Edit highlighted code via the `Ctrl+I` flow. Single-file scope. (Lesson 07.)
Agent	Read files itself, write files itself, run shell commands itself — gated by your approval. Multi-file.

Agent mode is the closest thing these open models have to "the AI is doing the work." It's also where a weaker model can wreck your repo, so you read every step.

Step 1: switch to agent mode

1. Open the Continue chat panel.

2. At the bottom of the panel, find the mode dropdown (or the toggle next to the model picker — the UI moved a few times). Switch to **Agent**.

3. The chat input now shows a hint about tool use. Some agent UIs show available tools (read, write, run) listed.

Step 2: a small agent task

4. Make sure the FastAPI repo is open and your tree is clean.

```
cd ~/docker-course/fastapi-app
git status
```

5. Create a feature branch (good hygiene — agent mode is going to make changes).

```
git switch -c agent-add-status-route
```

6. In Continue's agent chat, type:

```
Add a new route GET /status to the FastAPI app that returns:
{
  "uptime_seconds": <how long the process has been running>,
  "items_count": <number of items in the database>
}

Steps:
1. Find an appropriate file to add the route to (probably main.py or a new router).
2. Implement the route.
3. Show me the change before applying it.
4. After I approve, run a curl against /status to verify.
```

Send.

Step 3: watch and approve

7. The agent will start working. You'll see a series of **tool calls** — each one with an "Approve" button.

8. A typical sequence:

- `read main.py` — agent reads the file. Click **Approve**.
- `read routers/items.py` — agent learns the pattern. Click Approve.
- `propose edit to main.py` — agent shows a diff adding the `/status` route. **Read the diff.** If it looks right, Approve. If wrong, Reject and ask the agent to redo.
- `run curl http://localhost:8000/status` — **but the stack might not be running.** The agent might need to start it first.

9. Read each step. Some prompts you'll commonly see:

- "Should I run `docker compose up -d`?" — yes if your stack is down.

- "Should I write `models.py`?" — only if you understand why.
- "Should I run `rm <something>`?" — almost always **no**, then ask why.

10. If the agent makes a mistake (added an import to the wrong file, used a function that doesn't exist), reject the bad step and either:

- Type new instructions: `That import goes at the top of main.py, not in routers/items.py. Try again.`
- Or take over manually with `Ctrl+I` (lesson 07).

Step 4: verify

11. Once the agent reports done, manually verify.

```
docker compose up -d
sleep 3
curl -s http://localhost:8000/status | python3 -m json.tool
```

You should see `uptime_seconds` and `items_count` keys.

12. Look at what changed.

```
git diff
```

Read every line. Agent-generated code is no different from human-generated code — read it.

Step 5: commit and merge

13. If the change is good, commit. Otherwise revert.

```
git add -A
git commit -m "Add /status route via agent mode"
git switch main
git merge agent-add-status-route
git branch -d agent-add-status-route
```

Drill 2: a multi-file change

14. Switch to a new branch.

```
git switch -c agent-add-tags
```

15. Agent task:

```
Add a 'tags' field to items, stored as a comma-separated string in the DB. Steps:

1. Update the SQLAlchemy Item model to add `tags: str | None`.
2. Update the Pydantic ItemIn and ItemOut models to include `tags: list[str] |
None`.
3. In the create_item route, accept a list of strings and store them comma-joined.
4. In the read endpoints, split the stored string back into a list.
5. After applying, restart the stack and curl POST /items with a tags array, then
GET /items/<id> to verify the round-trip.

Show me each file change before applying.
```

16. Watch the agent. This is multi-file:

- `models.py` (new column)
- `routers/items.py` (Pydantic models + serialize/deserialize)
- Possibly a `docker compose down -v && docker compose up -d` to recreate the schema (the agent should suggest this — `Base.metadata.create_all` doesn't add columns to existing tables)

17. Read every diff. If the agent forgets to recreate the DB schema, the new column won't exist and POST will fail. Coach it:

```
The schema needs to be recreated. Run docker compose down -v then docker compose up
-d.
```

18. Verify with a real round-trip:

```
curl -s -X POST http://localhost:8000/items \
  -H "Content-Type: application/json" \
  -d '{"name":"Tagged","price":1.0,"tags":["red","sale"]}' | python3 -m json.tool
```

Then GET it back and confirm `tags` round-trips as an array.

19. Commit if good, revert if bad.

```
git status
git add -A
git commit -m "Add tags field to items via agent mode"
git switch main
git merge agent-add-tags
git branch -d agent-add-tags
```

When agent mode goes wrong

20. Common failure modes with an open model like the 70B:

Symptom	Cause	What to do
Agent loops trying the same broken edit	model lost track of state	New chat, smaller scope, more explicit steps
Agent invented a function name	hallucination	Reject, ask "Show me where that function is defined"
Agent wants to `rm` your files	aggressive cleanup	**Reject** every destructive command unless you explicitly asked
Agent never finishes the task	context filled up	New chat, narrower scope

21. Default to skepticism. The 70B is good for surgical edits and small features. It is not a frontier cloud model. For bigger tasks, decompose them yourself and use agent mode on each piece.

Step 6: log it

22. Log the lesson.

```
echo "Agent mode drilled: /status route + tags field on items" >> ~/docker-course/notes/notes.txt
```

Watch out

- **Approve every tool call deliberately.** "Yes to all" is a footgun. The whole point of the approve gate is human-in-the-loop.

- **Branch first.** Always `git switch -c agent-<thing>` before letting the agent make changes. Easy to revert if the result is junk.
- **Schema migrations.** SQLAlchemy's `create_all` only creates tables that don't exist — it won't add columns. If the agent changes a model, the DB needs to be recreated (lose data) or migrated (Alembic, future module).
- **Small scope wins.** Three-step agent tasks succeed often. Ten-step ones rarely do on these open models. Decompose.

Checkpoint

```
curl -s http://localhost:8000/status | python3 -m json.tool 2>/dev/null
```

Should print a JSON with at least `uptime_seconds` (or fail cleanly if you reverted — both are fine for the checkpoint).

```
git log --oneline -3
```

Should show your "via agent mode" commits.

```
git status
```

Should be clean.

Next up: 09-final-checkpoint.md — Ship a feature end-to-end with the local stack

Lab — do it for real. Complete every step above on your own machine — type the commands, don't copy-paste. **Submit:** a screenshot or pasted output of the *Checkpoint* section (or this lesson's final verification steps) Keep it. If you cannot submit in D2L, bring it to class or email it to your instructor. Completing the lab unlocks the next lesson.

Learning objective: Build a case-insensitive `GET /items/search` route with the Continue.dev + college vLLM stack, verify it with curl and Swagger UI, and ship it through a GitHub pull request.

Lesson 09: Final Checkpoint

Time: ~20 min — **no instructor help.**

Build a small feature using the full stack: Continue.dev + the college vLLM endpoint, against the FastAPI repo, shipped through a GitHub PR (Module 04 workflow).

The task

Add a **`GET /items/search?q=<query>`** route that returns all items whose `name` contains the query string (case-insensitive).

You decide how to use the AI — chat, autocomplete, inline edit, or agent mode. The point is to use the college stack to ship a real feature.

Constraints

- The route must be on the existing items router (`routers/items.py`).
- Use SQLAlchemy `ilike` or equivalent for case-insensitive matching.
- Return a list of `ItemOut`.
- An empty query (`q=` or missing) returns all items (or a 400 — your call, justify it in the PR).
- Test it with curl from the terminal **and** from `/docs`.
- Ship it through a feature branch + PR + merge on GitHub.

Suggested workflow

1. Branch.

```
cd ~/docker-course/fastapi-app
git switch main
git pull
git switch -c add-item-search
```

2. Use whichever Continue mode you prefer to build the route. **Read every diff before accepting.**

3. Restart the stack and verify with curl:

```
docker compose restart api
sleep 3
# create a couple of items first if needed
curl -s "http://localhost:8000/items/search?q=apple" | python3 -m json.tool
```

4. Open `/docs` and try the route from Swagger UI.

5. Commit, push, open a PR on GitHub, leave at least one self-review comment, merge, delete the branch.

```
git add routers/items.py
git commit -m "Add /items/search route"
git push -u origin add-item-search
# then PR + merge in the GitHub web UI
git switch main
git pull
git branch -d add-item-search
```

Acceptance test

After merging:

```
git log --oneline --graph -5
```

Should show the merge commit from your PR.

```
docker compose up -d
sleep 3
curl -s -o /dev/null -w "%{http_code}\n" "http://localhost:8000/items/search?
q=anything"
```

Should print `200`.

```
# create test data
curl -s -X POST http://localhost:8000/items -H "Content-Type: application/json" -d
'{"name":"Apple","price":1}' > /dev/null
curl -s -X POST http://localhost:8000/items -H "Content-Type: application/json" -d
'{"name":"Banana","price":2}' > /dev/null
curl -s "http://localhost:8000/items/search?q=app" | python3 -m json.tool
```

Should return one item (the apple), not the banana.

What to submit

A screenshot showing **all of these** in one image:

1. VS Code with the new code visible in `routers/items.py` and the Continue chat panel showing the prompt you used.
2. A terminal with the search curl returning matching items.
3. The GitHub PR page showing your PR as **Merged**.

Self-grade

You pass this module if:

- [ ] The search endpoint behaves as specified.
- [ ] You used Continue.dev (chat / inline edit / agent mode — any combination) and the college vLLM models to build it.
- [ ] The PR was opened, reviewed (one comment minimum), and merged on GitHub.
- [ ] You did **not** sign in to a paid cloud AI service — the free VS Code + Continue.dev + college endpoint stack did the whole job.

Then:

```
echo "Module 05 final checkpoint: passed $(date -Iseconds)" >> ~/docker-
course/notes/notes.txt
```

What's next

Module 06 (session 12) drills effective prompting and refactoring patterns against this same repo, same stack. The hard infrastructure work is done — from here it's about getting better at *using* the agent.

Lab — do it for real. Complete every step above on your own machine — type the commands, don't copy-paste. **Submit:** a screenshot or pasted output of the *Checkpoint* section (or this lesson's final verification steps) Keep it. If you cannot submit in D2L, bring it to class or email it to your instructor. Completing the lab unlocks the next lesson.